TECH DESIGN AND SOLUTION DOCUMENT

Coding Assessment Platform

Version 1.0 | Prepared for Capstone Project

Prepared By

Raghav Somasundaram PL

Field	Value
Project	Coding Assessment Platform
Document Type	Technical Design and Solutioning
Planned Duration	4 Weeks
Primary Stack	React, FastAPI, PostgreSQL, AI API, GCP
Date	08 June 2026

1. Summary

The Coding Assessment Platform is a web-based system that enables Recruiters to create a question bank, build coding assessments, invite candidates, conduct browser-based coding tests, evaluate submissions, rank candidates, and generate reports. The platform uses AI to accelerate question generation, difficulty classification, test case generation, reference solution generation, and code quality evaluation. The core business value is reducing manual assessment preparation effort while improving consistency, transparency, and speed in candidate evaluation.

1.1 Problems Being Solved

- Manual coding test preparation is time-consuming and inconsistent.
- Creating good edge cases and hidden test cases requires significant technical effort.
- Candidate evaluation based only on pass/fail does not explain code quality, approach, or complexity.
- Manual report preparation and ranking slows down decision-making.
- Candidate access should be simple; separate candidate account creation is unnecessary for assessment links.

1.2 Business Impact

- Faster assessment creation using AI-assisted question and test case generation.
- Standardized candidate evaluation using automated hidden test execution and AI feedback.
- Improved Recruiter visibility through live candidate progress monitoring and ranking reports.
- Reduced operational work through tokenized invitations, automated emails, and report generation.
- A deployable MVP architecture that can be extended into a production-grade assessment platform.

2. Requirements

2.1 Users

Recruiter

The person (user) incharge to conduct and manage the Coding assessment representing a management or a organisation to evaluate their candidates in our platform.

Eg. Hiring Manager of a Tech Company

Candidate

The users who attends the Interview or Placement drive in a management or a organisation to show out their coding : problem solving and logical thinking skills.

Eg. Student attending a placement drive

User	Purpose	Main Actions
Recruiter	Creates and manages assessments.	Login, manage questions, build assessments, upload candidates, send invites, monitor progress, view reports.
Candidate	Attends assigned coding assessment.	Open invite link, start test, write code, run samples, submit solution, view submission summary.

2.2 Use Cases and Success Criteria

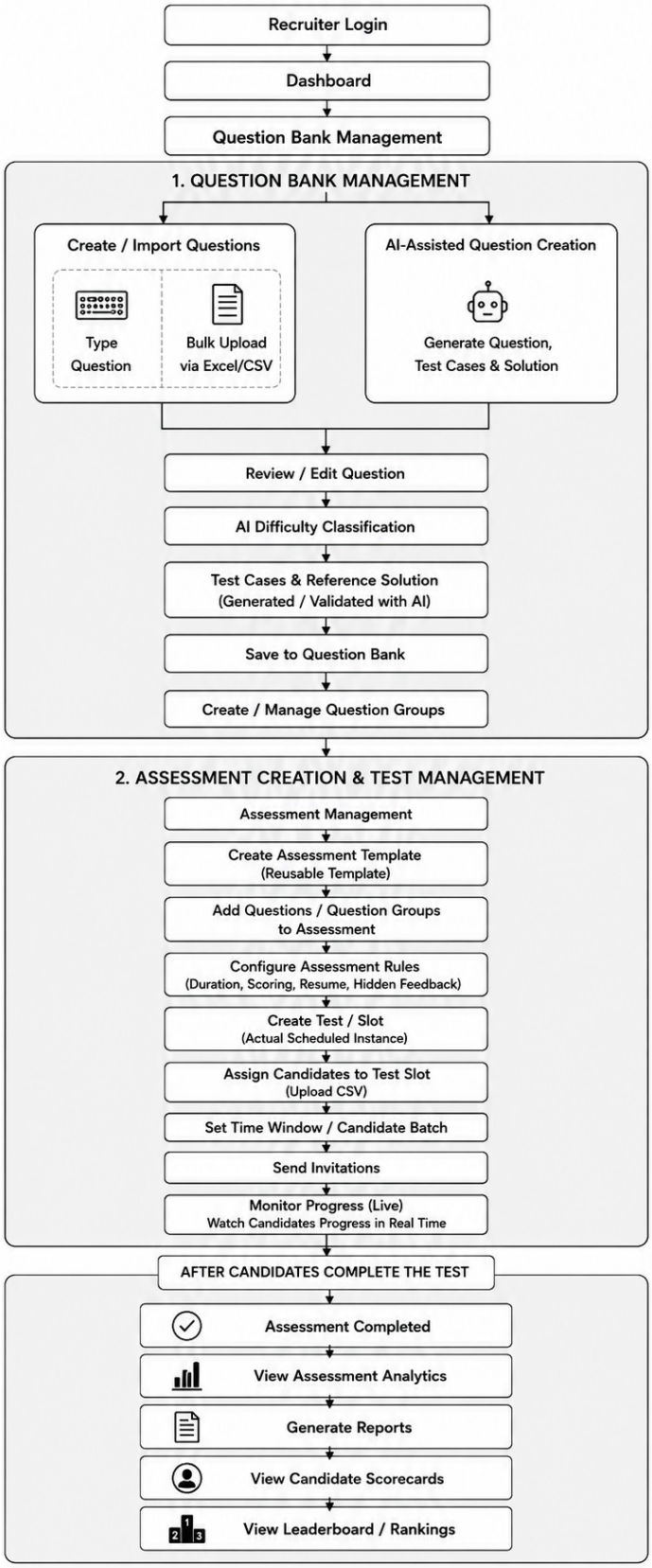
Description	Description	Success Criteria
UC-01 Recruiter Login	Management user logs into the Recruiter dashboard.	Valid credentials generate a secure session and unauthorized users are blocked.
UC-02 Question Bank	Recruiter creates, edits, filters, imports, and selects questions.	Questions are searchable, editable, tagged, and reusable.
UC-03 AI Question Pipeline	AI generates questions, test cases, difficulty, and reference solutions.	AI output is structured, reviewable, validated, and stored only after approval.
UC-04 Assessment Builder	Recruiter selects questions and configures the assessment.	Assessment is saved with duration, window, passing score, and selected questions.
UC-05 Candidate Invitation	Recruiter uploads candidates and sends email links.	Each candidate receives a unique tokenized invite and status is tracked.
UC-06 Candidate Test Portal	Candidate attends test in browser editor.	Candidate can run samples, save progress, navigate questions, and submit.
UC-07 Live Monitoring	Recruiter tracks candidates during test.	Status, attempted count, and submission progress are visible.
UC-08 Evaluation	System executes hidden tests and AI evaluates code quality.	Score, pass rate, errors, complexity, readability, and rating are stored.
UC-09 Reports	Recruiter views leaderboard and candidate reports.	Rankings and PDF reports are generated for assessment review.

2.3 User Journeys and Flows

2.3.1 Recruiter Flow

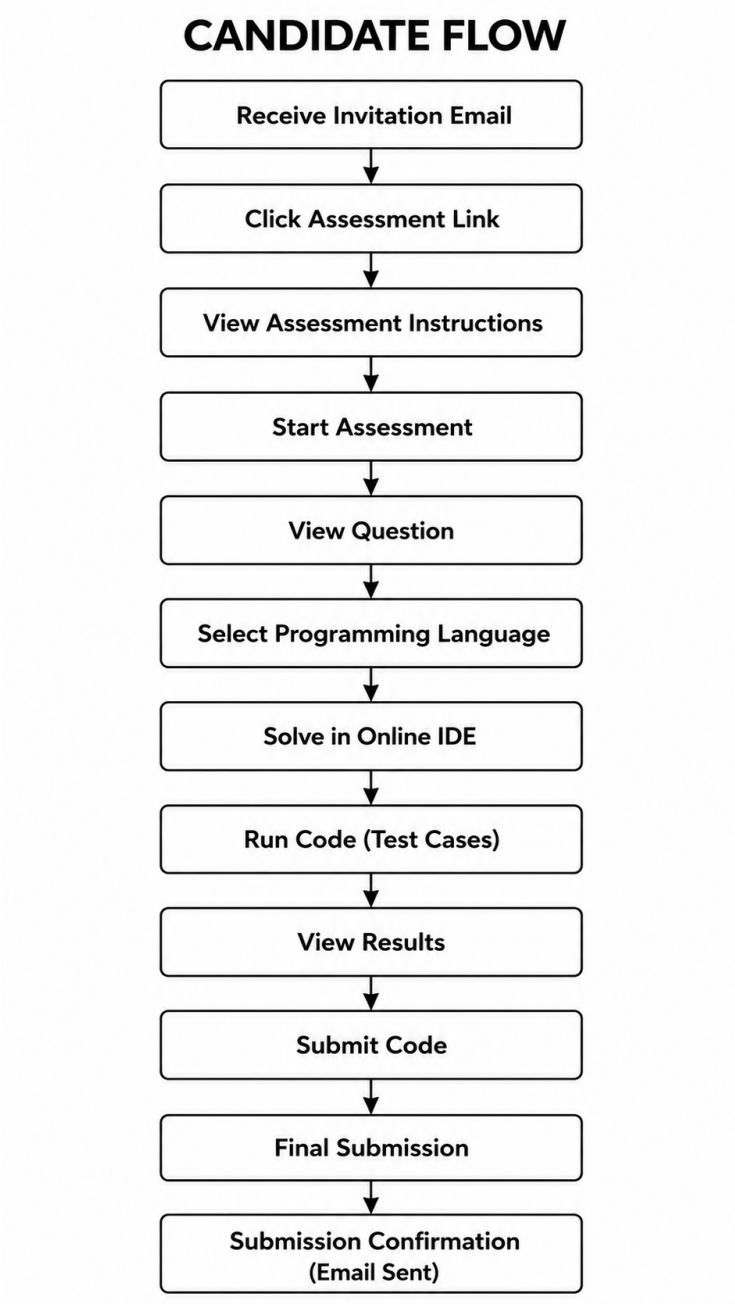
1. Recruiter signs up or logs in and accesses the recruiter dashboard.
2. Recruiter opens Question Bank Management to create, import, validate, and manage coding questions.
3. Recruiter adds questions using available options such as typing a question manually, bulk uploading questions through Excel/CSV, or using AI-assisted question creation.
4. Recruiter reviews and validates the question, difficulty level, reference solution, and test cases before saving it into the question bank.
5. Recruiter creates and manages Question Groups by grouping validated questions based on role, skill, topic, or difficulty.
6. Recruiter creates an Assessment Template by selecting questions or question groups and configuring rules such as duration, scoring weightage, resume policy, and hidden feedback.
7. Recruiter creates one or more Test Slots from the same assessment template, assigns candidate batches, sets the time window, uploads candidate lists, and sends email invitations.
8. Recruiter monitors candidate progress during the test and, after completion, accesses analytics, generated reports, leaderboard, rankings, and individual candidate scorecards.

RECRUITER FLOW



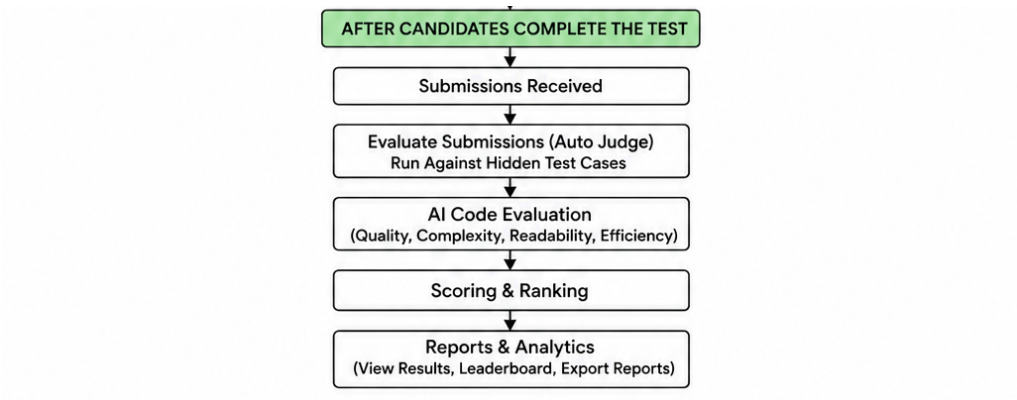
2.3.2 Candidate Flow

1. Candidate receives email invitation with secure assessment link.
2. Candidate opens the link and the backend validates the invite token.
3. Candidate views assessment instructions and starts the test.
4. Candidate writes code in the browser IDE and runs sample test cases.
5. Candidate submits solutions before timer expiry or the system auto-submits saved code.
6. Candidate sees a submission confirmation summary.



2.3.3 Post-Assessment System Flow

1. Submission is stored in PostgreSQL.
2. Backend sends code and hidden test cases to self-hosted code execution.
3. Code execution system returns execution status, stdout, stderr, compile output, time, and memory.
4. Backend compares output with expected output and calculates score.
5. AI evaluation agent reviews code quality, approach, complexity, readability, and improvement areas.
6. Final score and rank are calculated.
7. Reports and leaderboard are made available to the Recruiter.



2.4 Scope of the project

IN SCOPE
User Management • Management Login • Token-based Candidate Access • Basic Role-Based Access (Management / Candidate)
Question Bank • Create Question Manually • Edit/Delete Question • Search & Filter Questions • Question Tags • Difficulty Levels • Bulk Upload via Excel/CSV • Select Existing Questions

AI Question Generation • Generate Full Coding Question • Generate Test Cases • Generate Reference Solution • AI Difficulty Classification • Review & Approve Question
Assessment Management • Create Assessment • Add Questions • Add Candidates (CSV Upload) • Send Email Invitations • Assessment Status Tracking
Candidate Assessment Portal • Access via Secure Link • Instructions Page • Take Assessment • Monaco Text Editor & Compiler • Different Languages • Run Against Sample Test Cases • Timer • Question Navigation
Live Monitoring • View Candidate Status • Time Remaining • Questions Attempted • Submission Status
Evaluation Engine • Hidden Test Case Execution • Score Calculation • Approach Identification • Complexity Estimation • Readability Analysis • Code Quality Review • Overall Rating
Reports • Candidate Scorecard • Leaderboard • Candidate Ranking • Assessment Summary • Export PDF

OUT SCOPE

Proctoring

- Webcam Monitoring
- Screen Recording
- Browser Lockdown
- Tab Switching Detection
- Face Verification

Advanced Coding Platform

- Collaborative Coding
- Pair Programming
- Live Coding Interviews
- Whiteboard Interviewing

Integrations

- No integration with other coding platforms like Slack, HackeRank, Leetcode, LinkedIn, Teams.

Admin portal

- Admin user and portal

2.5 Assumptions and Constraints

Type	Assumption / Constraint
User	*The candidate attending the assessment is assumed to be the actual candidate who received the invite link.*
User	*Candidates are expected to solve the coding questions independently without using external coding agents, AI tools, or unauthorized help.*
User	*Candidates are expected to have a stable internet connection and use a laptop or desktop browser during the assessment.*
User	*Recruiters are responsible for verifying candidate details before sending assessment invitations.*
Data	*Uploaded candidate and question data must follow the required Excel/CSV format.*
Data	*Hidden test cases must be stored securely and must not be exposed to candidates.*
Data	*Candidate submissions, scores, reports, and assessment links must be stored securely.*
Technical	*The system will be developed as a 4-week MVP for capstone evaluation.*
Technical	*The MVP will focus only on the core workflow: question creation, assessment creation, candidate test-taking, code execution, evaluation, and reports.*
Technical	*The MVP will initially support only selected programming languages such as Python, Java, and C++.*
Authentication	*Recruiters will access the dashboard using login-based authentication.*
Authentication	*Candidates will access assessments using unique token-based invite links instead of creating accounts.*
Authentication	*Each invite token must be mapped to one candidate and one assessment only.*
Security	*API keys, database credentials, email credentials, and cloud secrets must not be exposed in frontend code.*
Security	*Advanced proctoring features such as webcam monitoring, screen recording, face verification, and browser lockdown are not included in the MVP.*
AI	*AI agents will be used for question generation, test case generation, solution generation, difficulty classification, and code quality evaluation.*
AI	*AI-generated content must be reviewed or validated before being finalized.*
AI	*AI APIs and LLM models may be limited by free-tier usage, rate limits, or quota restrictions.*
Email	*Email invitations depend on the availability and limits of the selected bulk email service.*
Code Execution	*Self-hosted Judge0 will be used for compiling and executing candidate code.*
Code Execution	*Candidate code execution must be handled through the backend and should not be directly exposed to the frontend.*
Deployment	*Free-tier, limited-access, and open-source tools will be preferred because this is a capstone project.*
Deployment	*CI/CD and GCP deployment activities can begin only after GCP access is provided.*
Business	*The project is intended to demonstrate technical skills, system design, and implementation knowledge.*
Business	*The system may be scaled into a production-grade platform in the future if required.*

3. Design

3.1 High Level Design

3.1.1 Architecture

The platform is split into independent logical services/components so that the MVP remains simple while still being scalable. React handles Recruiter and candidate interfaces. FastAPI handles authentication, business logic, AI orchestration, Judge0 integration, email dispatch, and reports. PostgreSQL stores all platform data. Judge0 runs on a separate GCP Compute Engine VM for code execution. Frontend/backend deployment can run on GCP Cloud Run, while secrets are stored in Google Secret Manager.

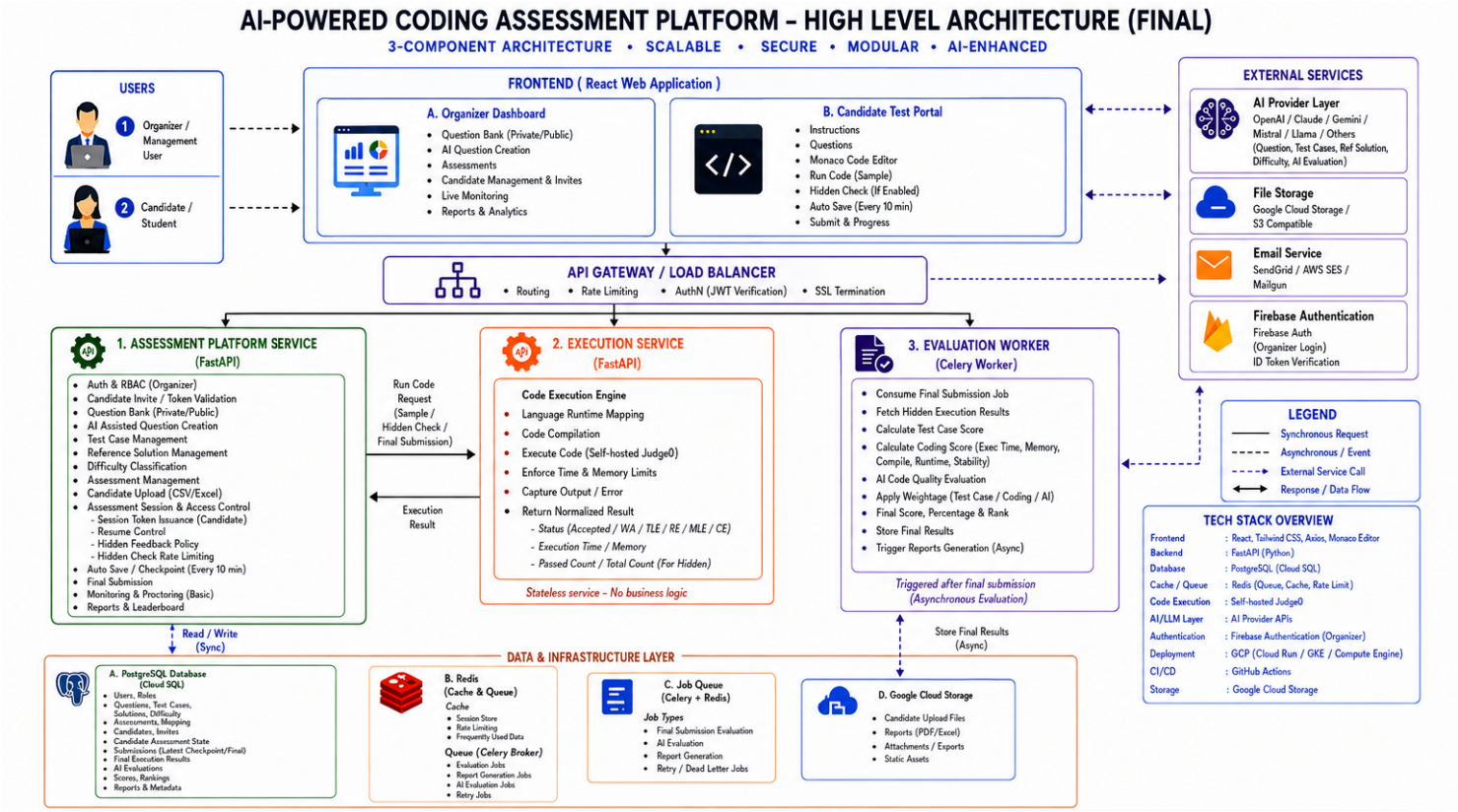
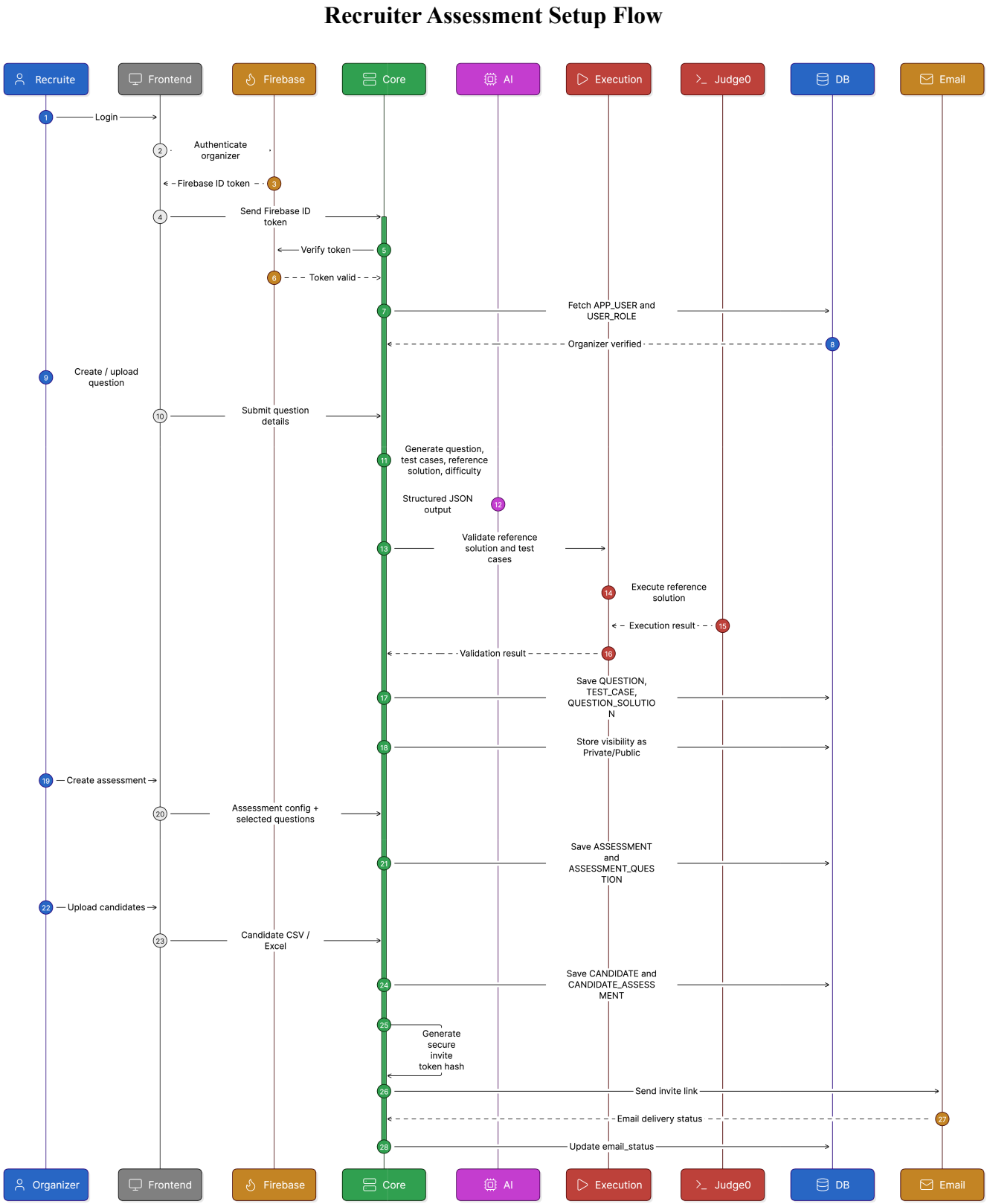
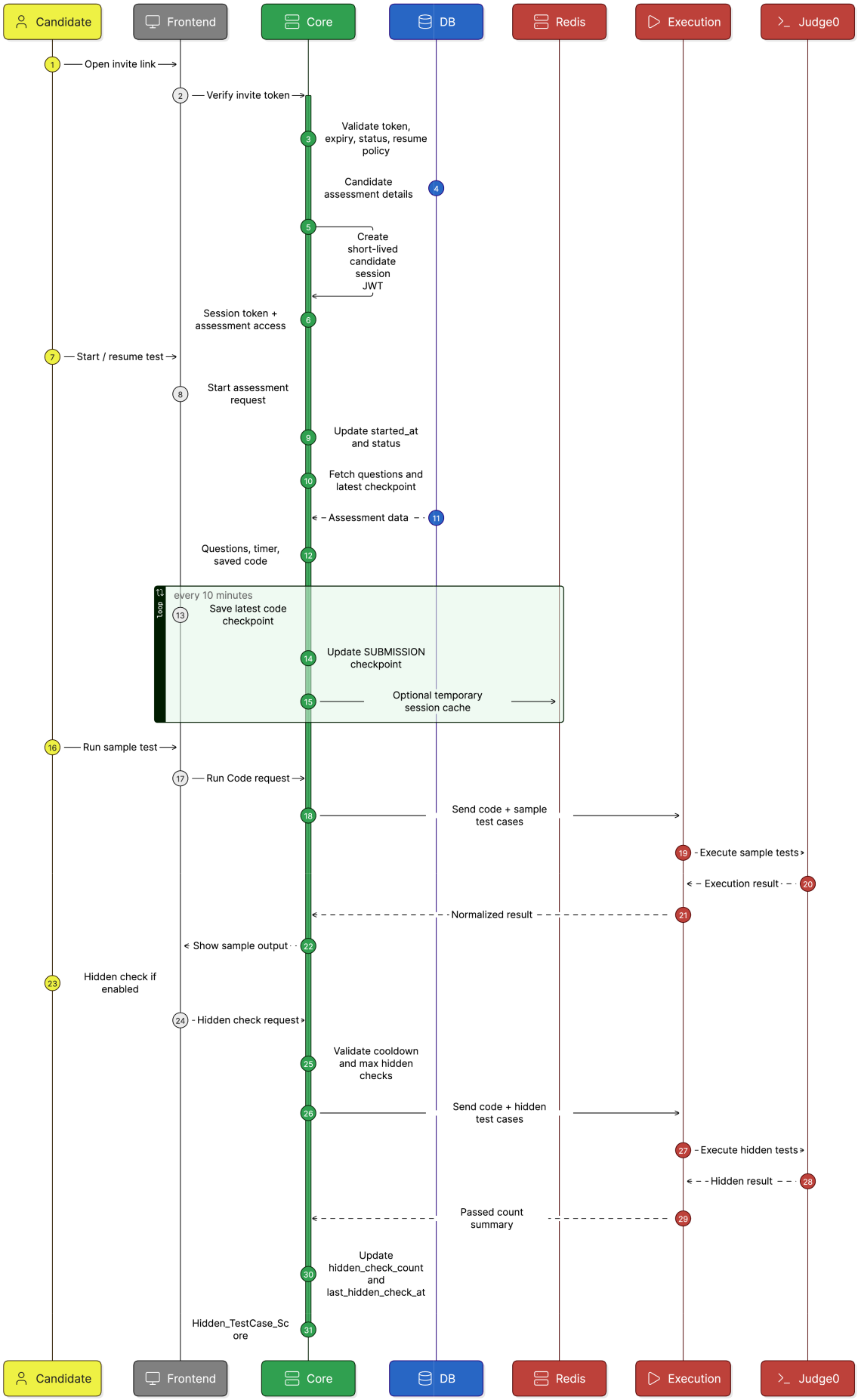


Figure 1: High-level architecture for the Coding Assessment Platform.

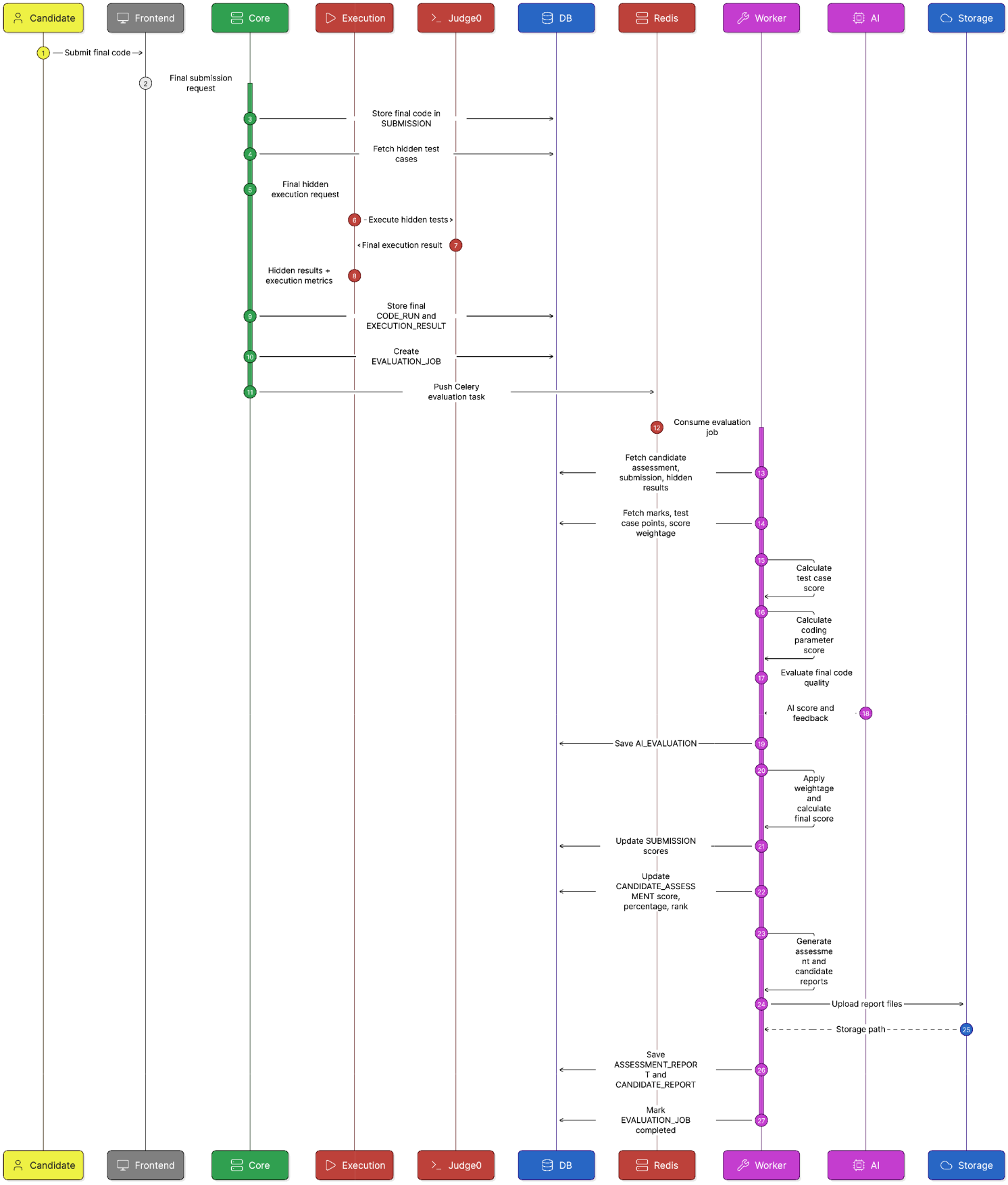
3.1.2 Sequence Diagram



Candidate Assessment Flow



Assessment Final Submission & Evaluation Flow



3.2 Major Components

Component	Responsibility	Technology
Recruiter UI	Dashboard, question bank, assessment builder, monitoring, reports.	React, TypeScript, Tailwind, TanStack Query
Candidate UI	Instructions, coding editor, run, submit, timer, navigation.	React, Monaco Editor, React Router
API Backend	Authentication, assessment logic, DB access, AI orchestration, Judge0 adapter.	FastAPI, Pydantic, SQLAlchemy
Database	Users, questions, assessments, candidates, invites, submissions, evaluations.	PostgreSQL
AI Structure	Common wrapper around AI models with JSON schema validation and retry.	API based Models
Code Execution	Compile/run candidate code against sample and hidden test cases.	Self-hosted Judge0 on GCP Compute Engine
Email Service	Send invitation emails and track delivery state.	SendGrid Web API, SMTP fallback
CI/CD	Automated build, test, Docker build, deploy.	GitHub Actions, GCP Cloud Run / Artifact Registry

3.3 Low Level Design / Detailed Design

3.3.1 Candidate Token-Based Login Design

Candidates should not create accounts for the MVP. A secure invite-token flow is simpler and safer for one-time assessments. The recommended approach is to use an opaque random invite token in the email link, store only its hash in the database, and issue a short-lived candidate session JWT after the link is validated.

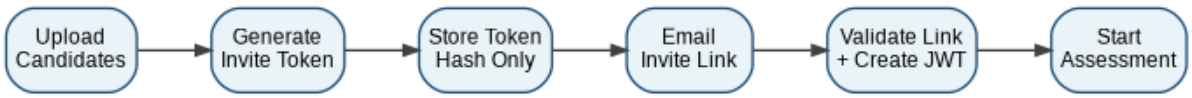


Figure 2: Candidate invite-token authentication flow.

Step	Design Decision
Token generation	Use a cryptographically secure random token, for example 32 bytes URL-safe.
Token storage	Store SHA-256 hash of token plus server-side pepper; never store raw token.
Email link	Send link like /candidate/invite/{raw_token}.
Validation	Hash received token, find invite, check assessment window, status, expiry, and candidate mapping.
Session creation	After validation, issue short-lived candidate JWT with role=candidate, invite_id, assessment_id, candidate_id.
Revocation	Recruiter can revoke an invite by changing invite status. Hash lookup makes revoked tokens invalid.
Resume behavior	Allow candidate to resume before final submission using same invite token or session refresh rules.
Security	Rate-limit token validation, always use HTTPS, hide whether a token belongs to a real email, and log suspicious attempts.

3.3.2 Email Invitation Design

For GCP deployment, the recommended email option is SendGrid Web API because Google Cloud blocks external SMTP port 25 in many cases, while ports 465 and 587 are allowed and third-party providers such as SendGrid is recommended options in Google Cloud documentation. For MVP, SendGrid Web API is preferred over raw SMTP because it provides an API-first integration and better tracking options.

Email Requirement	Design
Trigger	Recruiter clicks Send Invites after uploading candidate CSV.
Processing	Backend creates invite records and queues emails. For MVP, FastAPI BackgroundTasks is to be done.In future use Celery/Redis or Cloud Tasks.
Provider	SendGrid Web API using API key stored in Secret Manager.
Template	Assessment title, duration, deadline, instructions, unique invite link, support contact.
Status Tracking	Store pending, sent, failed, bounced, opened if event webhook is enabled later.
Retry	Retry failed sends with exponential backoff and show resend button to Recruiter.
Security	Never include hidden test cases or answers in email; invite link expires after assessment window.

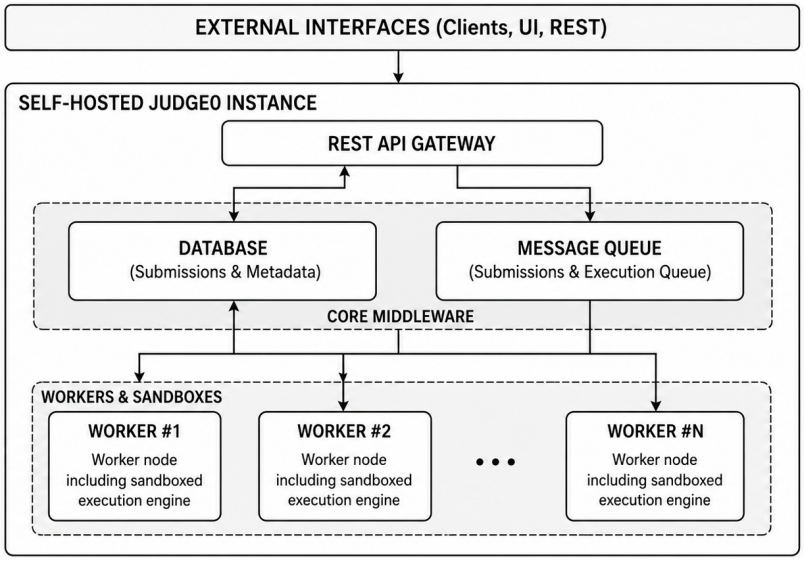
3.3.3 Code Execution Design

The platform will use self-hosted Judge0 instead of the public Judge0 API. Judge0 remains API-based, but the API runs on our own GCP Compute Engine VM. This keeps control within the project while avoiding the time and risk of building a custom execution sandbox from scratch.

Option	Pros	Cons	Decision
Self-hosted Judge0	Multi-language support, sandboxed execution, time/memory output, internal control.	Needs VM setup and monitoring.	Selected for MVP.

A self-hosted Judge0 instance can reduce common risks from user submissions by:

- Isolating code from the host through the sandbox layer.



Simplified Self-Hosted Judge0 Architecture Block Diagram

- Enforcing runtime controls such as **time limits, memory limits, and process/thread limits**, which helps stop infinite loops, memory abuse, and fork-bomb style behavior.
- Running submissions through a dedicated execution service instead of inside your FastAPI app, so your application code is not directly compiling or executing candidate code.

3.3.4 AI Agent Design and Model Selection

The AI layer should be implemented as an internal AI Gateway service, not directly spread across controllers. Each AI task must request structured JSON output, validate it with Pydantic, and store both the prompt version and model version for traceability. The recommended model strategy is to use free tier models as the primary path.

AI Agent	Requirement
Question Generation Agent	Needs strong reasoning and coding capability to produce well-structured problems.
Difficulty Classification Agent	High-volume, lower-risk classification task; speed and cost matter.
Test Case Generation Agent	Needs edge-case reasoning and structured JSON generation.
Reference Solution Agent	Code generation must be followed by compile-test-retry validation.
Code Quality Evaluation Agent	Evaluation must analyze approach, complexity, readability, inefficiencies, and rating.
Report Summary Agent	Report summaries are high-volume and less complex than solution generation.

3.3.5 Analysis of Approaches and Service-Level Technical Design

3.3.5.1 Assessment Platform Service

The Assessment Platform Service owns Recruiter workflows, assessment setup, candidate access, test session control, final submission, monitoring, and report access.

3.3.5.1.1 Authentication and Authorization

- Recruiter authentication is handled through Firebase Authentication.
- The frontend sends the Firebase ID token with every Recruiter API request.
- The backend verifies the Firebase token using Firebase Admin SDK.
- The verified firebase_uid is mapped with APP_USER.firebase_uid.
- Recruiter permissions are controlled using USER_ROLE.
- Candidates authenticate using a secure invite token generated and delivered through the assessment invitation email.
- After invite token validation, the backend creates a short-lived candidate session token (JWT) that is used to authorize all assessment-related API requests until submission or session expiry.

3.3.5.1.2 Question Bank Management

- Questions are stored in the QUESTION table.
- Each question stores title, description, difficulty, tags, supported languages, visibility, and status.
- Questions can be marked as **Private** or **Public**.
- Private questions are visible only to the creator.
- Public questions are available in the shared question bank for reuse.
- Sample and hidden test cases are stored in TEST_CASE.
- Reference solutions are stored in QUESTION_SOLUTION.
- Recruiters can create, edit, search, filter, upload, and reuse questions.
- Question status can be Draft, Reviewed, Active, or Archived.
- Public questions can be searched using tags, difficulty, and keywords.

3.3.5.1.3 Question Search and Similarity

- Questions can be searched using title, difficulty, tags, and supported languages.
- Tags help Recruiters quickly filter questions by topic such as arrays, strings, DP, graphs, or SQL.
- Similarity checking compares title, description, constraints, and tags.
- MVP will use keyword and tag-based similarity.
- Production can use embeddings to detect near-duplicate questions.
- Similarity score helps avoid repeated or copied questions in the question bank.

3.3.5.1.4 AI-Assisted Question Creation

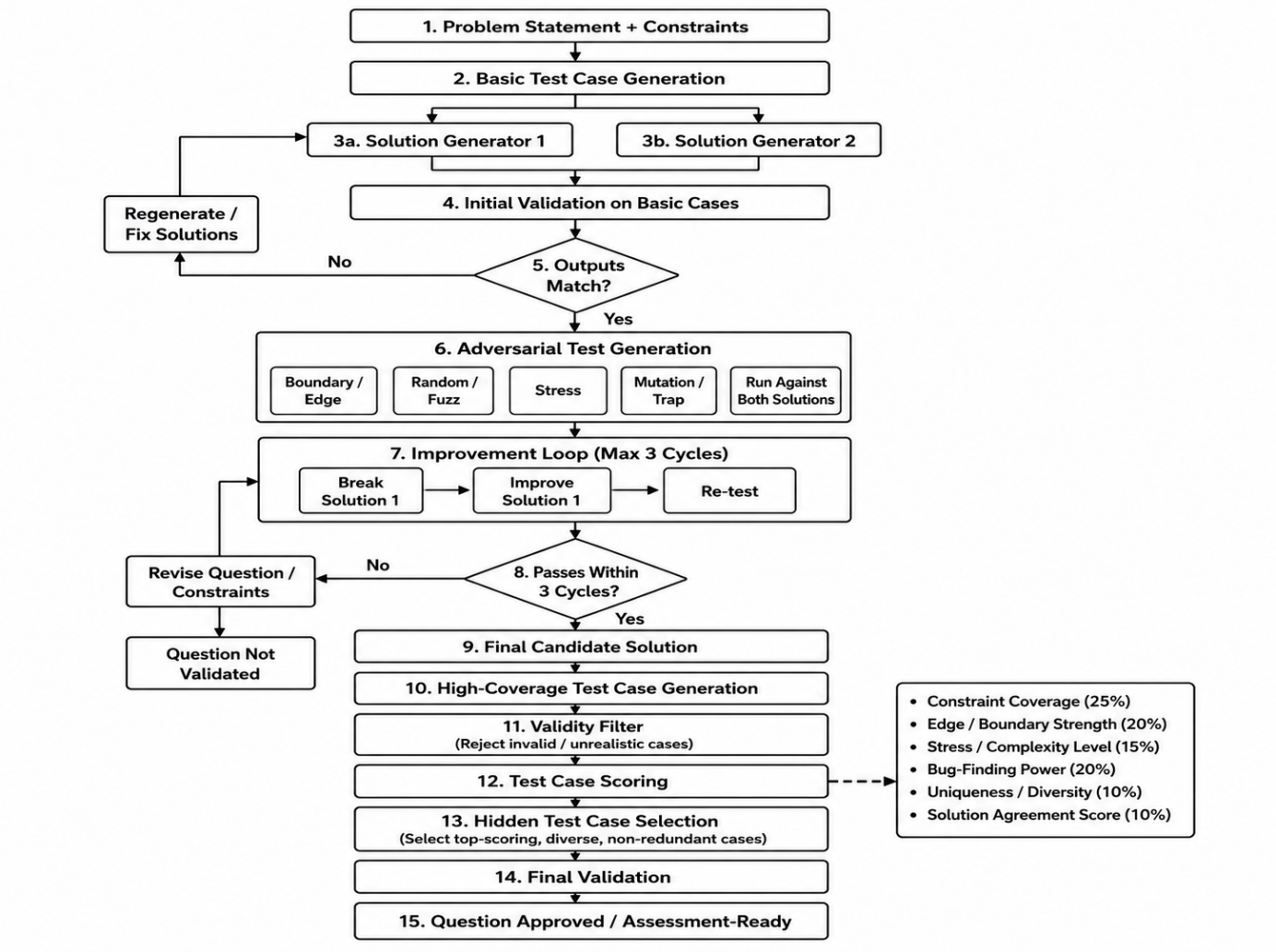
- AI generates draft questions, test cases, reference solutions, and difficulty classification.
- The AI request is sent using structured prompts.
- The AI response is expected in structured JSON format.
- AI output is not directly published.
- Recruiter reviews and edits the generated content.
- Approved AI output is stored in QUESTION, TEST_CASE, and QUESTION_SOLUTION.

3.3.5.1.5 Test Case Generation Strategy

- Test cases are pre-created before publishing the question, similar to online judge platforms.
- Recruiter can manually create test cases or use AI/script-assisted generation as a helper.
- Test cases are grouped as sample, basic correctness, boundary, edge, stress, and adversarial cases.
- Sample test cases are visible to candidates for understanding the problem.
- Hidden test cases are used for final evaluation and are not exposed to candidates.
- Expected outputs are generated by executing the trusted reference solution through Judge0.
- The complete test suite is validated by running the reference solution against all test cases.
- Weak, hardcoded, or inefficient solutions can also be tested to verify test case strength.
- Failed, duplicate, or inconsistent test cases are corrected before saving.
- Recruiter reviews and approves the final test suite before publishing the question.

3.3.5.1.6 Reference Solution Generation

- AI generates reference solutions for supported languages.
- Generated solutions are executed through Judge0.
- Solutions are validated against sample, edge, boundary, stress, and hidden test cases.
- Only 100% passing solutions are accepted.
- Failed solutions are regenerated with error feedback.
- Validated solutions are stored in `QUESTION_SOLUTION`.
- Recruiter reviews and approves before publishing.



3.3.5.1.7 Difficulty Classification

- AI classifies questions as Easy, Medium, or Hard.
- Classification uses logic, constraints, expected algorithm, complexity, and test coverage.
- Recruiter can trigger classification for manual or uploaded questions.
- AI gives suggested difficulty with reasoning.
- Recruiter can accept or override the difficulty.
- Final difficulty is stored in QUESTION.difficulty.
- Difficulty is used for search, filtering, assessment building, and reports.

3.3.5.1.6 Assessment Management

- Assessments are stored in ASSESSMENT.
- Questions are mapped to assessments using ASSESSMENT_QUESTION.
- ASSESSMENT_QUESTION stores question order, marks, time limit, and mandatory status.
- The same question can be reused in multiple assessments.
- Assessment configuration controls duration, start time, end time, passing score, and status.

- Recruiter publishes the assessment only after questions, candidates, and rules are configured.

3.3.5.1.7 Assessment Scoring Configuration

- Each assessment stores scoring weightage for final evaluation.
- Example: test case score 60%, coding parameter score 20%, AI score 20%.
- test_case_score_weight controls correctness-based scoring.
- coding_score_weight controls execution metric-based scoring.
- ai_score_weight controls code quality scoring.
- Total weightage must equal 100%.
- Final score is calculated only after applying configured weightage.

3.3.5.1.8 Hidden Test Feedback Policy

- Recruiter decides whether candidates can see hidden test feedback before final submission.
- Mode 1: no hidden feedback, where candidate sees no hidden result.
- Mode 2: summary-only feedback, where candidate sees only hidden tests pass score Eg. 6/10.
- Hidden input, expected output, actual output, and failed case details are never exposed.
- Hidden checks can be rate-limited.
- Example cooldown: one hidden check every 10 minutes.
- Maximum hidden tests check count can be configured per assessment.
- Hidden check count and last hidden check time are tracked in candidate assessment state.

3.3.5.1.9 Candidate Management

- Candidate data is uploaded using CSV or Excel.
- Valid candidate records are stored in CANDIDATE.
- Each candidate assigned to an assessment gets one CANDIDATE_ASSESSMENT record.
- CANDIDATE_ASSESSMENT stores invite token, email status, status, attempt number, start time, submit time, score, percentage, and rank.
- Duplicate candidate emails are validated before upload.
- Invalid candidate rows are rejected with error details.

3.3.5.1.10 Email Invitation

- Each candidate assessment gets a unique secure invite token.
- The invite token is embedded in the assessment invite link.
- Invite emails are sent using the configured email provider.
- Email delivery status is stored in CANDIDATE_ASSESSMENT.email_status.
- Recruiter can resend failed or pending invitations.
- Candidate cannot access the test without a valid invite token.

3.3.5.1.11 Candidate Assessment Session

- Candidate opens the assessment using the invite link.
- Backend validates invite token, assessment status, time window, expiry, and submission status.

- If valid, backend creates a candidate session token.
- Invite token identifies the candidate assessment.
- Session token controls active test access.
- Final submission invalidates the active candidate session.
- All candidate test APIs require a valid session token.

3.3.5.1.12 Resume Test Policy

- Recruiter can configure whether resume is allowed.
- If allow_resume = true, candidate can reopen the test before expiry and continue.
- Resume is allowed only if the candidate has started but not submitted.
- Resume is blocked if assessment time expired.
- If allow_resume = false, candidate cannot continue after leaving the session.
- If resume is disabled, Recruiter must manually reset the attempt if needed.
- Resume validation is enforced through invite token, session token, assessment status, and candidate status.

3.3.5.1.13 Code Draft Handling

- The main database stores only final submitted code permanently in `SUBMISSION`.
- Candidate code is auto-saved every 10 minutes to support resume and recovery.
- The latest auto-saved code is stored in `SUBMISSION` as a draft checkpoint.
- If resume is enabled, candidates can continue from the latest saved checkpoint.
- Code_run attempts and execution results are not stored permanently in the main database.
- Execution Service may temporarily store run logs for debugging with limited retention.
- Final submission overwrites the latest checkpoint and marks the code as submitted.
- Only the final submitted code is used for evaluation, scoring, ranking, and report generation.

3.3.5.1.14 Run Code Flow

- Candidate clicks Run Code to test code against sample test cases.
- Assessment Platform Service fetches only sample test cases.
- Code, language, sample tests, time limit, and memory limit are sent to Execution Service.
- Execution result is returned to frontend for display.
- Sample input, expected output, actual output, and errors can be shown to the candidate.
- Code_run results are not stored permanently in the main database but updated in the SUBMISSIONS as the latest code check-pointing.

3.3.5.1.15 Hidden Test Cases Check Flow

- Hidden check is allowed only if Recruiter enables hidden feedback.
- Backend validates cooldown time and maximum hidden check count.
- Candidate code is sent to Execution Service with hidden test cases.
- Candidate receives only pass count summary.
- Output: Number of passed test cases / Total number of hidden test cases passed.
- Hidden test input, expected output, actual output, and failure reason are never shown.
- Hidden check metadata is updated in CANDIDATE_ASSESSMENT.

3.3.5.1.16 Final Submission Flow

- Candidate final submission stores the final code in SUBMISSION.
- Final submission locks the answer for evaluation.
- Assessment Platform Service sends final code and hidden test cases to Execution Service.
- Final hidden execution results are stored for scoring.
- After hidden execution, an EVALUATION_JOB is created.
- A Celery task is pushed to Redis for background evaluation.
- Candidate cannot modify code after final submission.

3.3.5.1.17 Live Monitoring

- Recruiter monitoring is calculated from CANDIDATE_ASSESSMENT, SUBMISSION, and timestamps.
- Recruiter can view not started, in progress, submitted, and auto-submitted candidates.
- Recruiter can view questions attempted and hidden check count.
- Recruiter can view candidate start time and submit time.
- MVP can use polling APIs or SSE for this feature.

3.3.5.1.18 Report Access

- Assessment Platform Service does not generate final reports.
- It reads generated report metadata from ASSESSMENT_REPORT and CANDIDATE_REPORT.
- Recruiter can view overall assessment summary.
- Recruiter can view leaderboard.
- Recruiter can view candidate scorecard.
- Recruiter can download generated reports.
- Report files are stored in cloud storage and accessed through stored paths.

3.3.5.2 Execution Service

The Execution Service runs code safely using self-hosted Judge0 and returns normalized execution results.

3.3.5.2.1 Execution API

- Assessment Platform Service sends execution requests to Execution Service.
- Request contains language, source code, test cases, time limit, memory limit, and run type.
- Run type can be sample_run, hidden_check, or final_hidden.
- Execution Service returns a standard response format for all languages.
- Execution Service does not handle candidate identity, scoring, ranking, or reports.

3.3.5.2.2 Language Runtime Mapping

- Platform language names are mapped to Judge0 language IDs.
- Example: Python, Java, and C++ map to their Judge0 runtime IDs.
- Unsupported languages are rejected before execution.
- Language validation prevents invalid runtime requests.
- Supported language list is controlled by backend configuration.

3.3.5.2.3 Judge0 Integration

- Execution Service communicates only with self-hosted Judge0.
- For each test case, it creates a Judge0 submission.
- Judge0 executes the code in a sandboxed runtime.
- Execution Service waits for Judge0 result.
- Judge0 response is converted into the platform's standard verdict format.
- Frontend never calls Judge0 directly.

3.3.5.2.4 Sample Test Execution

- Sample execution is used only for candidate debugging.
- It runs code against visible sample test cases.
- Candidate can see input, expected output, actual output, and error messages.
- Sample run result is returned immediately to frontend.
- Sample run code are logged but not stored permanently in the main database.

3.3.5.2.5 Hidden Test Execution

- Hidden execution is used for hidden checks and final submission.
- Hidden test case details are received only from Assessment Platform Service.
- For hidden check, only pass count summary is returned if enabled.
- For final submission, detailed results are returned to Assessment Platform Service for scoring.
- Hidden input and expected output are never returned to frontend.

3.3.5.2.6 Execution Logs

- Execution Service may log run attempts for debugging and audit.
- Logs can include language, run type, status, execution time, memory usage, and code snapshot.
- Logs must have limited retention.
- Logs are not treated as the main source of candidate submissions.
- Main database stores only final submitted code.

3.3.5.2.7 Execution Limits

- Execution Service applies time and memory limit for each run.
- Output size is limited to avoid excessive logs.
- Timeout is returned if execution exceeds allowed time.
- Infinite loops are controlled using execution timeout.

3.3.5.2.8 Error Handling

- Compile errors are returned as compile error verdicts.
- Runtime exceptions are returned as runtime error verdicts.
- Wrong outputs are returned as wrong answer verdicts.
- Timeout is returned as time limit exceeded.
- High memory usage is returned as memory limit exceeded.
- Judge0 failures are converted into standard execution failure responses.

3.3.5.2.9 Execution Result Output

- Execution result includes verdict, stdout, stderr, compile output, execution time, and memory usage.
- Result also includes passed count and total test count.
- Sample results are returned to frontend.
- Hidden check results return only safe summary.
- Final hidden results are returned to Assessment Platform Service for scoring.
- Only final hidden execution results are stored permanently for evaluation.

3.3.5.3 Evaluation Worker

The Evaluation Worker runs asynchronously using Celery and Redis after final submission.

3.3.5.3.1 Celery and Redis Queue Design

- Assessment Platform Service creates EVALUATION_JOB after final hidden execution.
- A Celery task is pushed into Redis.
- Evaluation Worker consumes pending jobs from Redis.
- Job status is updated as pending, processing, completed, or failed.
- Celery retry handles temporary failures.

3.3.5.3.2 Evaluation Job Processing

- The worker fetches candidate assessment, final submission, hidden test results, and assessment configuration.
- Test case performance and execution metrics are collected for scoring.
- AI evaluates the final submitted code for quality and efficiency.
- Final score, percentage, rank, and evaluation status are calculated and updated.
- Assessment and candidate reports are generated.
- Evaluation job status is marked as Completed or Failed.

3.3.5.3.3 Test Result Aggregation

- Execution results are grouped by question.
- Passed and failed hidden test cases are counted.
- Test case points are summed question-wise.
- Mandatory test case failures can reduce or block marks if configured.
- Question-level correctness score is calculated.
- Candidate-level correctness score is aggregated.

3.3.5.3.4 Test Case Score Calculation

- Test case score is based on hidden test case pass results.
- Each hidden test case can carry equal or custom points.
- Passed test cases add points & failed test cases add zero points.
- Partial score is calculated question-wise.
- Final test_case_score is normalized to assessment weightage.

3.3.5.3.5 Coding Parameter Score Calculation

- Coding parameter score uses execution metrics.
- It considers execution time, memory usage, compile success, runtime stability, timeout and memory limit exceeded status.
- Candidate performance can be compared against reference solution benchmark.
- Final coding_score is normalized to assessment weightage.

3.3.5.3.6 AI Code Quality Evaluation

- AI evaluates only the final submitted code, approach correctness, time complexity, space complexity. readability, modularity and maintainability.
- AI identifies strengths, weaknesses and improvement suggestion of the candidate and list in scorecard.
- AI output is stored in AI_EVALUATION.

3.3.5.3.7 Final Score Calculation

- Final score combines test case score, coding parameter score, and AI score.
- Assessment weightage controls how each score contributes.
- Formula: final_score = test_case_score * test_case_weight + coding_score * coding_weight + ai_score * ai_weight.
- Final score is stored in SUBMISSION.final_score.
- Candidate total score is stored in CANDIDATE_ASSESSMENT.total_score.
- Percentage is stored in CANDIDATE_ASSESSMENT.percentage.

3.3.5.3.8 Candidate Score Update

- Test case score, coding score, AI score, and final score are updated in SUBMISSION.
- Candidate total score, percentage, and evaluation status are updated in CANDIDATE_ASSESSMENT.
- Candidate rank is recalculated based on the updated final score.
- Assessment leaderboard is refreshed with the latest results.

3.3.5.3.9 Rank Calculation

- Ranking is based on weighted final score.
- First tie-breaker is higher test case score.
- Second tie-breaker is better coding parameter score.
- Third tie-breaker is lower execution time.
- Fourth tie-breaker is lower memory usage.
- Fifth tie-breaker is lower time taken to submit.
- Sixth tie-breaker is earlier submission time.
- Final rank is stored in CANDIDATE_ASSESSMENT.rank.

3.3.5.3.10 Report Generation

- Worker generates overall assessment report.
- Worker generates candidate-wise report as scorecard.
- Overall report includes leaderboard, score distribution, question-wise performance, pass statistics, average time, and average memory.
- Candidate report includes final score, test case score, coding score, AI score, question-wise score, and AI feedback.
- Report files are uploaded to storage.
- Overall report metadata is stored in ASSESSMENT_REPORT.
- Candidate report metadata is stored in CANDIDATE_REPORT.

3.3.5.3.11 Failed Evaluation Retry

- Failed jobs are marked as failed.
- Error message is stored in EVALUATION_JOB.error_message.
- Retry count is updated in EVALUATION_JOB.attempt_count.
- Celery can retry temporary failures automatically.
- Recruiter can manually trigger re-evaluation if needed.
- Successful retry updates score, rank, and reports.

3.4 APIs / Contracts

3.4.1 Internal API Groups

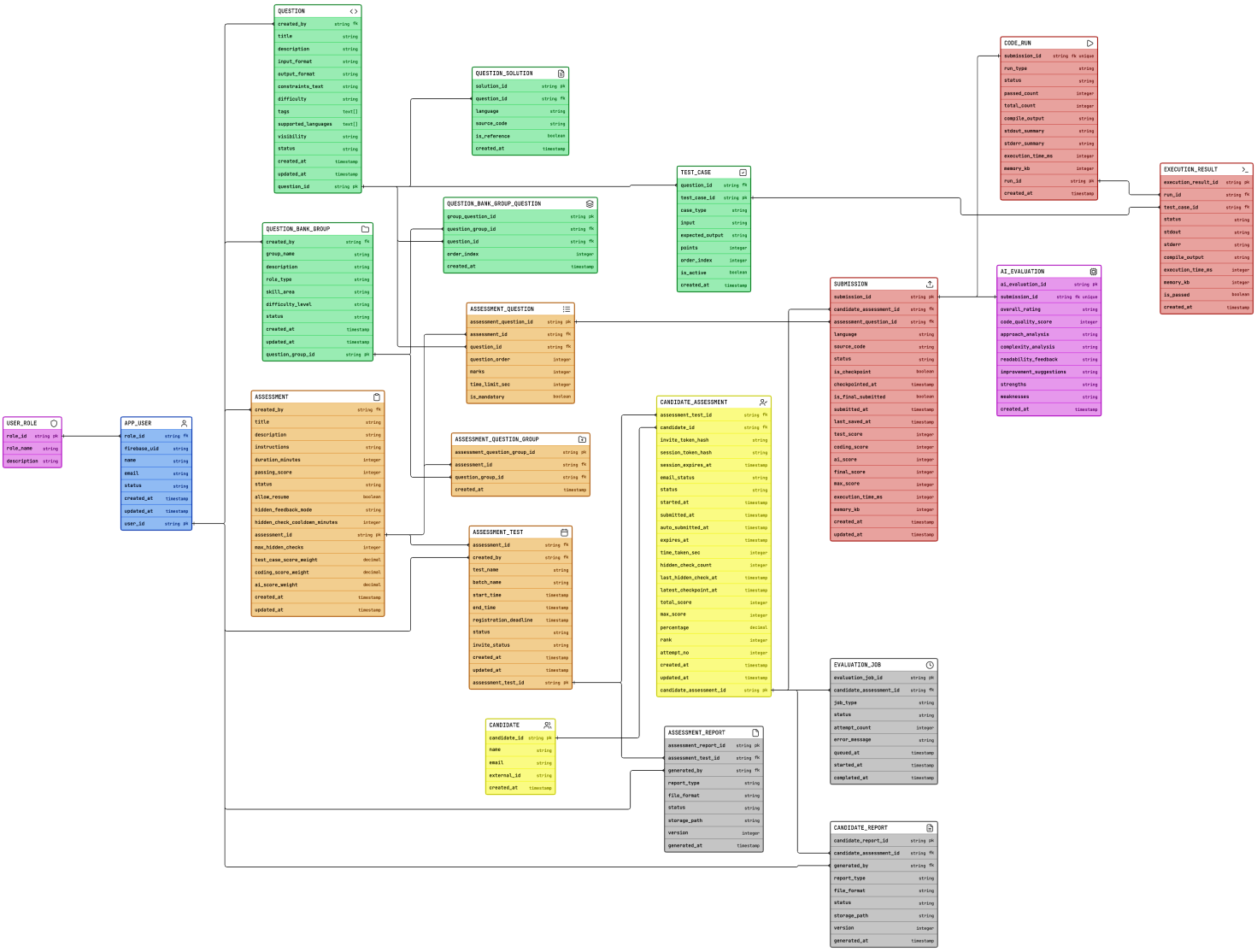
API Group	Endpoint Examples	Purpose
Recruiter Auth APIs	`POST /api/auth/verify-firebase-token`	Verify Firebase ID token, map Recruiter to APP_USER, and apply role-based access.
Question Bank APIs	`POST /api/questions`, `GET /api/questions`, `PUT /api/questions/{id}`	Create, edit, search, filter, upload, and reuse private/ public questions.
AI Question APIs	`POST /api/ai/questions/generate`, `POST /api/ai/test-cases/generate`, `POST /api/ai/reference-solution/ generate`, `POST /api/ai/difficulty/classify`	Generate question drafts, test cases, reference solutions, and difficulty classification.
Assessment APIs	`POST /api/assessments`, `PUT /api/assessments/{id}/publish`, `POST /api/assessments/{id}/questions`	Create assessments, configure rules, add questions, set scoring weights, and publish.
Candidate APIs	`POST /api/candidates/upload`, `POST /api/assessments/{id}/invite`	Upload candidates, map candidates to assessments, generate invite tokens, and send emails.
Candidate Session APIs	`POST /api/candidate/verify-invite`, `POST /api/candidate/start`, `POST /api/candidate/checkpoint`	Validate invite token, create session JWT, start/resume test, and save 10-minute code checkpoint.
Code Execution APIs	`POST /api/execution/sample-run`, `POST /api/execution/hidden-check`, `POST /api/execution/final-hidden`	Run sample tests, optional hidden summary check, and final hidden test execution through Execution Service.
Evaluation APIs	`POST /api/evaluation/jobs`, `GET /api/evaluation/jobs/{id}`	Create Celery evaluation jobs, track status, retry failed evaluations, and process scoring.
Monitoring APIs	`GET /api/assessments/{id}/monitoring`	Show candidate status, start time, submit time, questions attempted, and hidden check count.
Report APIs	`GET /api/reports/assessment/{id}`, `GET /api/reports/candidate/{id}`	Access overall assessment reports, leaderboard, candidate scorecards, and downloadable files.

3.4.2 Authentication / Authorization

Role	Auth Method	Allowed APIs
Recruiter	Firebase Authentication + Firebase ID token verified by backend	Question bank APIs, AI generation APIs, assessment APIs, candidate upload APIs, invite/email APIs, monitoring APIs, report APIs
Candidate	Secure invite token from email + backend-generated short-lived candidate session JWT	Assessment access APIs, question loading APIs, checkpoint save APIs, sample run APIs, hidden check APIs if enabled, final submit APIs

3.5 Data Models

The data model is relational because assessments, questions, candidates, invites, submissions, and evaluations have strong relationships and transactional requirements.



3.6 Failure Handling

Failure Scenario	Handling Strategy
Invalid/expired candidate token	Show generic invalid-link message and ask candidate to contact Recruiter. Do not reveal whether email exists.
Email send failure	Mark email_logs failed, retry with backoff, allow Recruiter to resend.
AI API failure	Allow manual creation and retry AI generation later. Store failed generation status.
Malformed AI output	Validate with Pydantic, reject response, retry with schema error context.
Judge0 unavailable	Queue evaluation, mark submission pending_evaluation, retry automatically.
Candidate network issue	Autosave code every 10-15 seconds; allow resume until assessment expires.
Timer expiry	Auto-submit latest saved code with status auto_submitted.
Database failure	Return safe error, log trace ID, do not lose candidate code if autosave already succeeded.
GCP deployment issue	Fallback to local deployment/demo environment until CI/CD is restored.

3.7 Testing Approach

Testing Type	Scope
Unit Testing	Token generation/validation, scoring logic, output normalization, AI response parsing.
API Testing	Auth APIs, question APIs, assessment APIs, candidate APIs, evaluation APIs using Postman/pytest.
Frontend Testing	Recruiter flow, candidate flow, timer, navigation, run/submit states.
Integration Testing	FastAPI + PostgreSQL + SendGrid sandbox + Judge0 + AI Gateway.
Load Testing	Simulate candidates running sample tests and submitting solutions.
UAT	Recruiter creates assessment; candidates complete test; reports generated end-to-end.

3.8 Non Functional Requirements

NFR Area	Requirement
Reliability	Critical candidate operations should be idempotent where possible; retries for email, AI, and Judge0 calls.
Security	HTTPS, hashed passwords, hashed invite tokens, JWT expiry, role-based APIs, hidden tests protected.
Privacy	Candidate PII not disclosed; no unnecessary AI sharing; logs avoid raw tokens and passwords.
Observability	Structured logs, request ID, error traces, evaluation job status, email send status.
Scalability	MVP supports minimum concurrent candidates; queue-based evaluation will be done.
Maintainability	Modular FastAPI routers/services/repositories; AI Gateway and Judge0 adapter isolated.
Compliance	Inform candidates about data usage; retain submissions only as per Recruiter policy.
Cost Control	Restrict enabled languages, limit AI retries, cap Judge0 resources, use Flash models for high-volume tasks.

4. Onboarding

4.1 Prerequisites to External Systems

System	Prerequisites
GCP	Project access, billing enabled, Cloud Run, Compute Engine, Artifact Registry, Secret Manager.
PostgreSQL	Local PostgreSQL for dev; Cloud SQL or managed PostgreSQL for deployment.
Judge0	GCP Compute Engine VM with Docker and Docker Compose.
SendGrid	Account, API key, verified sender/domain, optional event webhook.
AI Provider	API key, model access, rate limits checked.
GitHub	Repository, branch protection, Actions secrets, deployment credentials.

4.2 Detailed Onboarding Process

1. Create GitHub repository and set branch protection rules.
2. Set up local backend, frontend, and PostgreSQL.
3. Create database schema and run migrations.
4. Configure SendGrid sandbox/local mail testing, then verified sender for deployment.
5. Set up self-hosted Judge0 on a GCP Compute Engine VM and test /languages and /submissions endpoints from backend only.
6. Configure AI provider keys and verify structured JSON generation.
7. Dockerize frontend and backend.
8. Configure GitHub Actions CI/CD to build, test, push images, and deploy.
9. Store secrets in GitHub Actions secrets and GCP Secret Manager.
10. Run end-to-end test: Recruiter creates assessment -> candidate receives email -> candidate submits -> report generated.5. Launch Plan

5.1 Timelines

Week	Theme	Key Deliverables
Week 1	Design Document & Foundation + Question Bank + AI Basics	Project setup, auth, question CRUD, Excel/CSV upload, AI question generation, difficulty classification.
Week 2	Assessment Builder + Email + GCP/CI-CD	Assessment builder, candidate upload, invite tokens, SendGrid integration, Docker, GCP setup, CI/CD start.
Week 3	Candidate Portal + Code Execution	Candidate invite flow, Monaco editor, timer, autosave, run samples, self-hosted Judge0 integration.
Week 4	Evaluation + Reports + Final Deployment	Hidden test evaluation, AI feedback, ranking, PDF reports, bug fixes, final deployment, demo.

6. References

1. https://www.researchgate.net/publication/346751837_Robust_and_Scalable_Online_Code_Execution_System
Došilović, Herman Zvonimir & Mekterovic, Igor. (2020). Robust and Scalable Online Code Execution System. 1627-1632. 10.23919/MIPRO48935.2020.9245310.